

AI GRAND CHALLENGE 2025

PROBLEM STATEMENT 12 FORM

1. Briefly describe your proposed solution and its key features.

- The system detects four underwater sound categories and outputs clean, whole-second, non-overlapping events per PS-12 rules. It uses a compact CRNN: two convolutional blocks learn robust spectral patterns from log-mel spectrograms, and a bidirectional GRU stabilizes predictions over time for long, continuous acoustic events. Training optimizes a class-balanced focal loss to counter skew toward vessels and background-like segments; a light temporal median blend on logits reduces frame spikes without blurring short activity. Validation tracks an event-level proxy that converts frame posteriors to nearest-second, non-overlapping segments, aligning early stopping with how the organizer scores submissions.
- At inference, each audio is converted to 64-mel log spectrograms (n_fft 2048, hop 1024, 20–20 kHz). The CRNN produces per-frame posteriors, optionally median-smoothed; scores are aggregated to per-second maxima and fused using three cues: per-second argmax, adaptive quantile thresholds with class-specific floors/ceilings, and a vessel energy gate from a 50–3000 Hz band-pass with a robust RMS test. One class is chosen per second. De-chatter absorbs isolated 1-second flips into stronger neighbors, and vessel continuity fills small gaps and lightly extends edges when evidence is strong. Very low-score segments are pruned. The final JSON is COCO-style with integer start/end seconds, duration consistency, and global non-overlap enforced and validated.

2. Describe your understanding of the specific challenges and nuances of the selected problem.

- Underwater audio is long-duration, non-stationary, and often low SNR. Engines can be intermittent amid swell, rain, and other man-made noises, while marine animal calls are sparse and easily masked. A frame-wise detector tends to oscillate around class boundaries, producing 1-second chatter that inflates identification errors when submissions must be non-overlapping and rounded to whole seconds. Handling the organizer's contract—single class per second, integer times, no overlaps—means the post-processing must be metric-aware, not just a thresholding afterthought.
- Domain drift adds another challenge: gain and noise characteristics vary by deployment, and prior calibration may not transfer. A per-audio calibration (temperature probing and min-max stretching) helps align score distributions so thresholds behave consistently across files. Class imbalance is real: vessel tends to dominate, while "marine_animal" and "other_anthropogenic" can be underrepresented; a class-balanced focal objective keeps learning signal for minority classes without collapsing everything into vessel.
- Event continuity matters: GT-like labels typically prefer long stretches with few transitions, but real spectrograms contain micro-gaps, harmonics, and cross-energy that confuse models. Using a band-pass energy prior for vessel clarifies where sustained mechanical energy exists; de-chatter removes isolated flips; and small continuity fills avoid unnecessary fragmentation. These choices reduce misclassified seconds at boundaries.
- Finally, engineering constraints: memory on CPU requires on-the-fly features, small batches, and modest context windows during training, while still evaluating with second-wise eventization. Strict schema validation and deterministic sorting prevent formatting rejections and keep outputs reproducible under submission rules.

3. Please describe the architecture of your end-to-end solution. How are the components (reading of data files and preprocessing) organized in your dockerized setup?

- Ingest: The container mounts an input directory of 48 kHz WAVs and a model artifact. A file enumerator builds the “audios” list with id, file_name, path, and duration from the WAV headers.
- Feature extraction: Each WAV is read in mono and transformed into a 64-mel log spectrogram (n_fft 2048, hop 1024, fmin 20 Hz, fmax 20 kHz). No dataset-wide caching is required; features are computed on the fly to keep RAM stable.
- Model inference: The CRNN (2× Conv2D blocks → BiGRU(128×2) → Linear(4)) runs on CPU, returning per-frame logits; sigmoid gives per-frame posteriors. Optional median smoothing reduces frame noise.
- Per-audio calibration: A fast temperature probe and per-class min–max stretch align score ranges for that file so later thresholds behave consistently. (If temperature is disabled, quantile-based adaptive thresholds still stabilize decisions.)
- Second-wise fusion: For each second, take per-class maxima and fuse three votes: (a) per-second argmax, (b) adaptive thresholds with floors/ceilings per class, and (c) a vessel energy gate using a 50–3000 Hz band-pass with a robust RMS test. One class is chosen per second; ties are resolved by posterior strength.
- Event shaping: De-chatter reassigns isolated 1-second flips when neighbors are stronger; vessel continuity fills short gaps (up to ~12 s) and lightly extends edges where evidence is strong; very low-score segments are pruned.
- JSON writer and validation: Segments are emitted as non-overlapping, integer-second annotations with duration=end–start, score in , and categories 1–4. A schema check enforces keys, id order, durations, and ranges; annotations are sorted by audio/time.
- Outputs: A single COCO-style JSON ready for upload, plus console stats by category for quick inspection.

Inside the docker image, the entrypoint script loads environment variables for input/output paths, ensures the model file is present, runs inference over all WAVs, and writes the validated JSON. The container exposes a simple CLI (e.g., --wav_dir, --model_path, --out_json) for batch jobs.

4. What acoustic features are extracted from the audio for sound activity detection and classification?

- Log-mel spectrograms: 64 mel bands, n_{fft} 2048, hop 1024, 48 kHz mono, f_{min} 20 Hz, f_{max} 20 kHz, log power with small epsilon. These capture broadband vessel energy, animal calls, and natural noise in a compact, robust representation well-suited to CNNs.
- Derived per-file energy: For vessel fallback, a 50–3000 Hz band-pass and robust RMS thresholding (median + $Z \cdot \text{MAD}$) mark seconds with strong mechanical energy, improving continuity on clear engine spans.

5. What ML/DL models are used for detecting for sound activity detection and classification in acoustic data?

- Detector: A CRNN comprising two Conv2D blocks (ReLU, BatchNorm, MaxPool), a bidirectional GRU (hidden 128 per direction), and a linear head producing 4 logits per frame. CNN captures spectral motifs; GRU provides temporal context for sustained events.
- Training: Class-balanced focal loss ($\gamma=1.5$) with weights [1.0, 2.0, 1.5, 2.0], AdamW optimizer, light temporal median regularizer over logits, and early stopping on a nearest-second, non-overlap validation proxy.
- Inference logic: Per-frame posteriors $\rightarrow$ per-second maxima $\rightarrow$ fusion of argmax, adaptive thresholds, and energy gate $\rightarrow$ de-chatter and continuity rules $\rightarrow$ pruned, non-overlapping segments.

6. How does your system handle .wav files of different sampling rate, gain and bit depth?

- Sampling rates: Audio is resampled to 48 kHz mono at load time, normalizing time–frequency mapping across files.
- Gain variation: Per-audio calibration (temperature probe + per-class min–max stretch) reduces sensitivity to global level differences; log-mel scaling further stabilizes dynamic range.
- Bit depth: Reading via soundfile/librosa abstracts bit depth; downstream features operate in float32. Robust thresholds and adaptive quantiles limit sensitivity to quantization differences.

7. In what format does your system output results (e.g., JSON, CSV, visualization)?

- COCO-style JSON with keys info, audios, categories, annotations. Categories use ids 1–4 with names: vessel, marine_animal, natural_sound, other_anthropogenic. Annotations have integer start_time/end_time, duration=end–start, and score in , globally non-overlapping per audio.

8. How do you propose to address requirements of future stages which would involve sub-classes among the main classes of Stage-1 and multiple overlapping classes at the same time?

- Sub-classes: Expand the head to more outputs and retain the second-wise fusion, using class-specific adaptive thresholds. For closely related sub-classes, add a hierarchical decision layer: primary class first, then sub-class disambiguation using additional cues (e.g., band-limited energy or narrowband harmonicity).
- Overlapping events: Switch from single-label per second to multi-label per second by removing the final “one-class” constraint and keeping independent threshold masks per class. Retain de-chatter per class and apply cross-class non-destructive merging. Evaluation-aware formatting can emit parallel tracks or multi-label second tags as per the new spec.

10. Describe any security measures implemented within your solution to protect data integrity and privacy, especially when handling sensitive data. Additionally, discuss any ethical considerations related to the use of AI/ML for Underwater Domain Awareness

- Data integrity: Read-only mounts for input audio; checksums for model files; schema validation before writing outputs; deterministic sorting and id ordering to prevent accidental duplication.
- Privacy: No PII is ingested or stored; outputs contain only acoustic event metadata. Intermediate features are kept in memory and not persisted unless explicitly enabled.
- Ethics: The system aids environmental monitoring and maritime safety. It avoids attributing intent, and it does not attempt vessel identification. Results are calibrated but not absolute; recommended use includes human oversight, especially in enforcement contexts.

11. Specify your requirements for computing resources (CPU, GPU), software and data access.

- Compute: CPU-capable for inference; training benefits from GPU but can run on CPU with windowed batches.
- Software: Python, PyTorch, librosa, soundfile, NumPy, SciPy. Containerized entrypoint accepts input paths and model artifacts, runs inference, and writes JSON.
- Data access: The container mounts an audio directory; no external network is required. Model weights are provided as a local file.

12. Describe the potential scalability of your solution.

- The pipeline is embarrassingly parallel at the file level: each WAV is processed independently, so horizontal scaling across cores, nodes, or containers is straightforward. Feature extraction is on-the-fly, avoiding large temporary datasets. The CRNN is lightweight and runs in real time or faster on CPU for typical file lengths; batching multiple files or frames increases throughput where memory allows.
- Post-processing is linear in seconds and uses simple array ops, so it scales proportionally with audio duration. The JSON writer streams results per file and validates structure without holding all audio in memory at once. If future stages expand to more classes or overlaps, the same architecture extends by widening the head and treating per-class decisions independently, while keeping quantile thresholds and de-chatter cheap. This maintains a low memory footprint and predictable runtime as datasets grow.

13. Describe your startup's previous projects that are relevant to this problem statement.

- Our team has delivered applied audio-ML systems that transform raw waveforms into reliable, second-wise event decisions under challenging acoustic conditions. Previous work focused on robust preprocessing (gain normalization, resampling consistency, and log-mel feature standardization), score calibration, and low-latency inference graphs. These pipelines handled heterogeneous recording setups, dynamic noise floors, and variable bit depths—challenges similar to underwater acoustic variability in PS-12.
- We have also built multimodal fusion and generative-audio models requiring careful alignment of heterogeneous signals, conflict resolution between model posteriors, and post-processing to suppress fragmented detections. Techniques such as adaptive per-class thresholds, median filtering, continuity rules, and non-overlap enforcement informed our approach to second-rounded eventization. This experience directly supports minimizing identification errors while preserving event boundaries in COCO-style submissions.
- Additionally, we have operated real-time ML architectures using ONNX-optimized CRNNs, batching strategies for short audio windows, and reproducible deployment practices (artifact versioning, automated validation, deterministic builds). These patterns translate to PS-12 by ensuring consistent second-wise outputs at scale, stable inference across diverse clips, and engineering resilience under submission constraints.